Building tx sets for parallel processing

Soroban transactions contain all the necessary information to enable building the transaction sets that can be applied in parallel and with bounded execution time. This document proposes a set of requirements for such partitioning and a simple partitioning algorithm that fulfills these requirements with reasonable performance.

Terminology

Here are some terms this doc uses:

- I/O Conflict (or just ‘conflict’) - when two transactions access the same mutable ledger entry, they can’t be run in parallel, which we call an I/O conflict. For Soroban footprints we have two kinds of conflicts:
    - RO-RW conflict - one transaction has an entry key in RO footprint and the other one has this same key in RW footprint
    - RW-RW conflict - both transactions have the same keys in the RW footprints
- Conflict cluster (or just cluster) - a connected graph, where nodes are transactions and edges are conflicts

Requirements

General requirements

These are requirements that are highly likely to be necessary for any feasible partitioning approach.

- Respect the inclusion fees - transactions with higher inclusion fees have to be prioritized over transactions with lower inclusion fees *within the same ‘fee lane’*, even if that leads to less optimal ledger utilization or lower total tx set fees, as much as possible
    - Not respecting this requirement might make it impossible for some users to get included into ledger under certain circumstances
    - This is how both classic and Soroban tx set building logic operates currently, even though the impact for Classic is really marginal
    - The ‘fee lane’ caveat is important to allow for scenarios similar to CAP-42 - making certain categories of transactions to compete between each other with a lower limit
    - ‘As much as possible’ means that sometimes the alternative is to either take a lower fee and lower resource transaction, or not take any transaction at all. For such situations it makes more sense to include the transaction, even though there is a higher fee transaction that has been skipped
- Bounded runtime - there should be an explicit upper bound on the apply time for all the transactions in the set
    - Due to validator hardware differences, the upper bound can only be defined in terms of modeled instructions.

    - Given that transactions are applied in parallel using multiple cores, it's only reasonable to define the upper bound for some 'minimum' validator configuration (e.g. 8 or 16 physical cores).
    - Validators with more cores may (but don't have to) apply the tx set faster, and validators with less cores are expected to apply it slower
    - There is no strict requirement for the slower validator runtime upper bound
- Deterministic validation - every validator should be able to validate the transaction set (including the upper runtime bound defined above), and every validator has to come to exactly the same conclusion about validity

Proposed requirements

These are additional, more restrictive requirements that are potentially more debatable than the ones listed above.

- Strict runtime upper bound - the transaction set should define the exact apply schedule for `N` threads (where `N` is the number of cores in minimum validator configuration), where every one of `N` threads should have transactions with up to `K` instructions total (where `K` is the respective network setting)
    - The alternative here could be e.g. to just pass the conflict clusters, every one of which is not larger than `K` instructions, and the total sum of instructions is not higher than `N * K`. This approach still comes with a runtime upper bound, but it might fluctuate between `K` and `2 * K` instructions, depending on the cluster sizes (worst case is `N + 1` cluster, each with `N * K / (N + 1)` instructions - this leads to `2 * K * N / (N + 1) ~= 2 * K` instructions necessary on `N` core machine).
    - Strict upper bound significantly reduces the apply time variance, which is already somewhat noisy for Soroban due to metering model imperfections.

Partitioning algorithm

The proposed algorithm consists of two relatively independent components: transaction application schedule structure, and the algorithm to build that schedule given the set of mempool transactions. The schedule building algorithm can be changed or tweaked without a protocol change, which makes it relatively easy to iterate on if necessary.

Apply schedule structure

The following structure is proposed for the 'parallel Soroban' phase of transaction set:

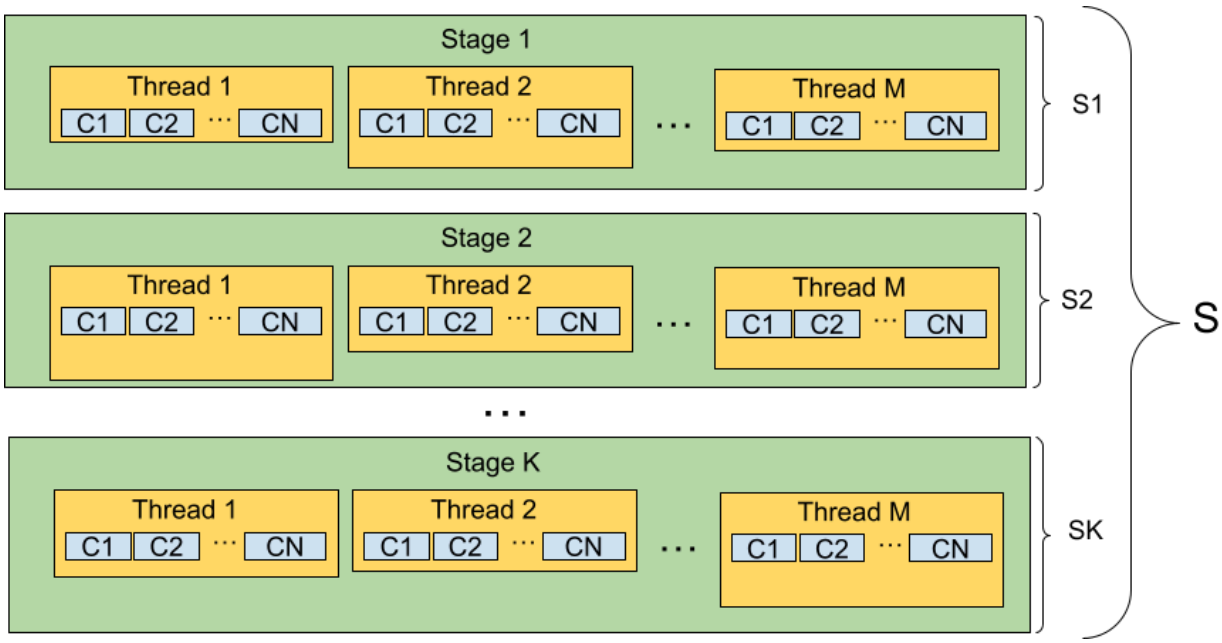
- Phase consists of an arbitrary number of 'stages'
    - Each 'stage' consists of up to `max_threads` non-conflicting 'logical threads': groups of transactions such that there is no pair of transactions that both have a conflict, and belong to different threads
        - Each thread consists of an arbitrary number of conflict clusters

- Every stage is considered to have execution time equal to the maximum sum of modeled instructions among its threads, i.e. the modeled runtime of the stage on a machine with at least `max_threads` cores
- Sum of modeled instructions across all the stages should not exceed `max_sequential_instructions`
- `max_threads` and `max_sequential_instructions` are network configuration parameters. The number of stages per transaction set could technically also be limited by a network setting as well, but that doesn't seem necessary as long as the transaction set follows all the validity criteria

In order to apply the transaction set, Core should execute the stages one-by-one, sequentially. Within a stage, every logical thread in the transaction set should be executed in a separate physical thread. As an optimization for machines with significantly more than `N` CPU cores (e.g. `2N`), it's possible to directly use the conflicting clusters in combination with a thread pool and ignore the provided thread partitioning.

The order of applying the sequential steps (stage order and order of transaction within a conflict cluster) should be randomized using a deterministic seed, similar to how we randomize the application order currently.

Here is the conceptual diagram illustrating the structure:



K - number of stages

M - number of threads, limited by `max_threads`

C*I* - inner conflict cluster in the thread structure (N is used just for convenience; it's different for every thread)

S*I* - Maximum instructions in a thread for the respective stage

S - Sum of S*I* across all the stages, limited by `max_sequential_instructions`

X-axis represents items that can run in parallel (clusters/threads in a stage), Y-axis represents the sequential flow (stages have to run sequentially). Taller boxes represent possible runtime differences (for threads/stages only, clusters may of course have different runtime as well).

Discussion

The proposed structure fulfills the strict runtime upper bound requirement (for an `N` core machine), but it also allows for potential speedup on more powerful machines. The speedup is not guaranteed by the protocol though - in the worst case a single cluster may use up the whole thread capacity in every stage.
The sequential stages allow implementing algorithms that work around I/O conflicts, at least to some degree. For example, if 100 transactions read an entry and a single transaction writes it (e.g. an oracle updates its feed), the write could be put in one stage, while all the reads could happen in different stages in parallel.
Flexible stage sizes allow for more algorithms to be implemented without changing the protocol. This limits the amount of shuffling that can be done during application: if every stage was forced to have the same size, then thread transactions could be shuffled across stages. However, since conflict clusters are anyways shuffled within the stage (as part of their cluster), the chances of introducing meaningful bias in application order are low.

Greedy partitioning algorithm

For the initial implementation of Soroban parallel execution we propose a straightforward greedy algorithm that creates a reasonable parallel application schedule given mempool transactions. The algorithm follows naturally from the inclusion fee requirement and the structure of the transaction set:
Inputs:

- `txs` - set of Soroban transactions
- `stage_count` - a parameter that defines the number of output stages
- `max_threads` - network setting, maximum number of threads per transaction set
- `max_sequential_instructions` - network setting, maximum number of declared instructions to allocate to a single thread (i.e. runtime upper bound)

Output:

- `txset` - the Soroban phase of transaction set that consists of `stage_count` stages, each stage containing at most `max_threads` logical threads, every thread limited to at most `floor(max_sequential_instructions / stage_count)` instructions

Algorithm (conceptually):
Main loop

- Prepare the data structure that resembles the final transaction set: `stage_count` stages, each with `max_threads` empty logical threads and `floor(max_sequential_instructions / stage_count)` instruction limit
- Iterate `txs` in descending inclusion fee order

    - Check that adding the transaction to the set doesn't exceed overall ledger resource limits (I/O limits, tx size limit etc.).
        - Skip the transaction if adding it would cause any limit to be exceeded
    - Try to add the transaction to an arbitrary stage that can fit it without violating the invariants defined by apply schedule structure above
        - Skip the transaction when there is no such stage

Adding transaction to a stage

- Find all the clusters in the stage that have an I/O conflict with the transaction being added
- Merge the conflicting clusters as well as the new transaction into a single new cluster
    - If the cluster size exceeds the stage instruction limit, then transaction is rejected and the stage state is not updated
- If the new cluster fits into one of the threads without excluding the per-thread instruction limit, simply move the whole new cluster into that thread
- If the new cluster doesn't fit into any one of the threads, run approximate bin packing on all the clusters in the stage (e.g. first-fit-decreasing) in order to try to reorganize the clusters into threads with the instruction limit.
- If bin packing didn't succeed, then the transaction is rejected and the stage state is not updated. Otherwise, store the newly packed clusters in the logical threads

Analysis

- The algorithm ensures that if two transactions have an identical footprint and instruction count, then the one with the higher inclusion fee will be included
    - In general, for the transactions with the same or similar conflict sets, the transaction with the higher fee will be prioritized in most cases, with the exception of the cases when a lower fee transaction has a lower instruction count, and in every stage the conflict clusters for that transaction are near the instruction limit. This behavior is similar to how we build Soroban transaction sets currently
- Evaluating the approximation ratio on the algorithm as a whole is pretty hard and probably not necessary. The approximation factor for FFD greedy bin packing is 11/9, which is the lower bound for the whole algorithm.
- Complexity is a bit tricky; it's roughly `O(N * F * C + T * B * C * log C)`, where `N` is the number of transactions, `F` is the maximum transaction footprint size, `C` is the number of clusters per stage, `T` is the number of threads, and `B` is the number of times we need to re-run the bin-packing. Value of `C` depends on the traffic patterns and in the worst case equals the number of transactions per stage (when every transaction is independent of every other transaction). Value of `B` depends on the traffic patterns as well and goes up with larger clusters and lower `C`.
    - In practice, a simplified prototype implementation without any optimizations and with `B == N` (i.e. re-running bin-packing every time), takes under 100 ms for 400 transactions/8 threads on a laptop. This definitely can be optimized to run a few times faster

Evaluation

As Soroban doesn't have much traffic yet, it's hard to predict what exactly the real traffic will be. Thus several synthetic benchmarks have been performed, covering various hypothetical scenarios.
Feel free to skip to summary for the result interpretation.

General benchmark setup

- `max_sequential_instructions` is set to 500M (matches the current ledger-wide instructions limit)
- `stage_count` is set for values from 1 to 4 (with per-stage limits of 500M to 125M respectively)
- `max_threads` is set to 8
- `2 * stage_count * max_threads * max_sequential_instructions` are generated (referred to as `N` generated transactions below) (i.e. 2x of total maximal per-ledger capacity).
- Generation assumes that the transaction queue doesn't try to do anything to reduce the number of conflicts between transactions.
- For most of the benchmarks per-transaction instructions are normally distributed with mean of 20M instructions and variance of 5M. This roughly resembles the mean instructions per transaction we observe now. For some uniform distribution between 500K and 100M instructions has been used. Instruction counts do not correlate with the footprints in any of the benchmarks.
- For all benchmarks, measure the utilization of the resulting transaction set (i.e. how much of the potential capacity of `` `stage_count * max_threads * max_sequential_instructions` `` has been used)
- Inclusion fees are generated uniformly at random between 100 and 1'000'000
- Various conflict patterns have been used (detailed in sections below)

Spreadsheet with all the data is available.

Randomized traffic

Build a randomized graph with transactions as nodes connected with an edge representing a conflict. Interpreting properties of random graphs is really tricky, so we generate the conflict clusters instead:

- Choose `conflicts_per_tx` - average number of conflict clusters per transaction.
- Generate conflict clusters in the following fashion, until `conflicts_per_tx * N` transactions have been included into a cluster:
    - Draw a value from Poisson distribution with `mean_ro_entries_per_conflict` mean and add the same key K to their RO footprint of that many transactions

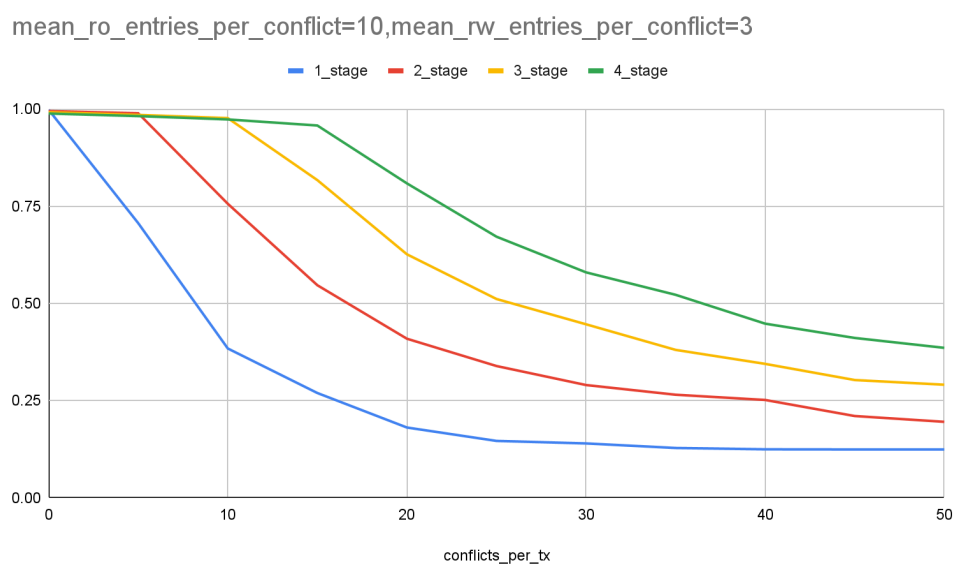
- Draw a value from Poisson distribution with `mean_rw_entries_per_conflict` mean and add the same key K to their RW footprint of that many transactions (excluding transactions where K is in RO footprint)

For example, if `conflicts_per_tx` is 10 and `mean_rw_entries_per_conflict` is 5, then on average every transaction has (5 - 1) * 10 = 40 RW-RW conflicts with other transactions *non-transitively*. As values of the input parameters grow, we'll get more transitive conflicts, but on the other hand more of the direct conflicts will be 'duplicate' (i.e. conflict for a same pair of transactions, but on a different key).

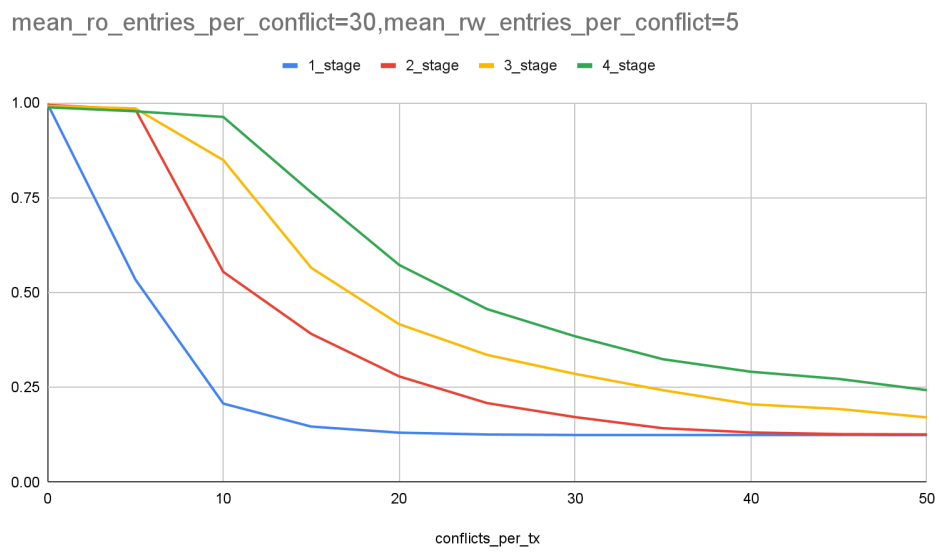
Benchmarks

Relatively small clusters (13 transactions on average) with RO-RW conflict bias: `conflicts_per_tx` =0..50, `mean_ro_entries_per_conflict=10,mean_rw_entries_per_conflict=3`

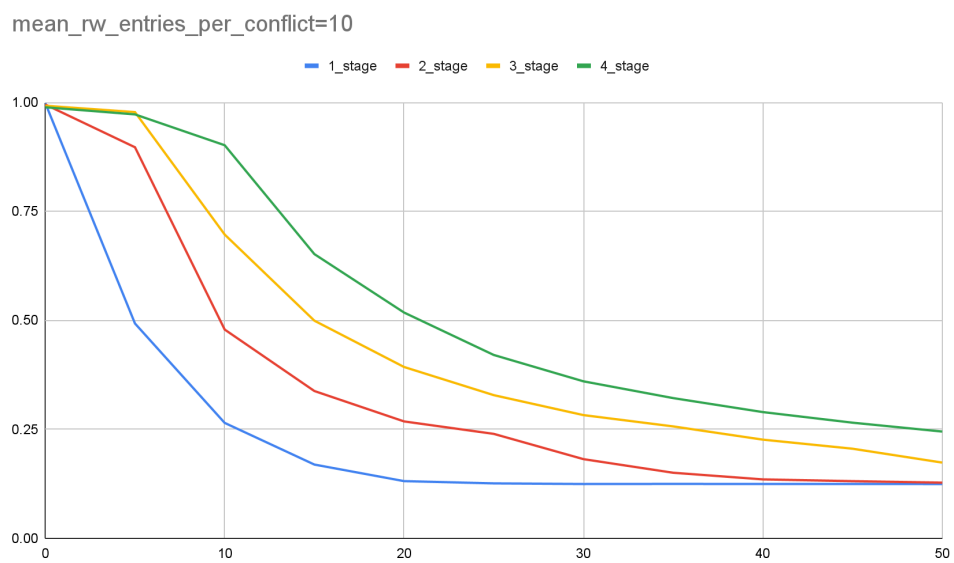
Note: All the plots have 'transaction set utilization' as Y axis, i.e. the fraction of maximum transaction set instructions that is used by the transactions.



Relatively large clusters (35 transactions on average) with RO-RW conflict bias: `conflicts_per_tx` =0..50, `mean_ro_entries_per_conflict=30,mean_rw_entries_per_conflict=5`



Smaller clusters (10 transactions on average) with RW-RW conflicts only: `conflicts_per_tx` =0..50, `mean_ro_entries_per_conflict=0`,`mean_rw_entries_per_conflict=1`



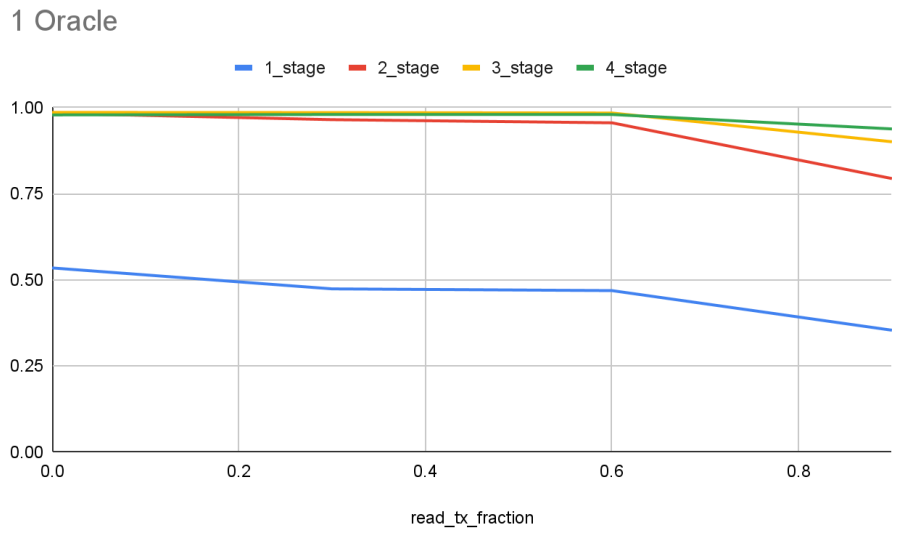
More scenarios are available in the [spreadsheet](#).

“Oracle update” scenario

This scenario emulates scenarios similar to updating an oracle contract. There are lots of readers of the same key and a single transaction that writes the key. A similar scenario is an update of a popular contract’s instance.

For the benchmark use the ‘baseline’ random traffic with relatively low contention and parameters `conflicts_per_tx=5`,`mean_ro_entries_per_conflict=30`,`mean_rw_entries_per_conflict=5`. Then add one more conflict with one write of the key with a high bid (to guarantee the inclusion) and `read_tx_fraction` fraction of transactions that read that key.

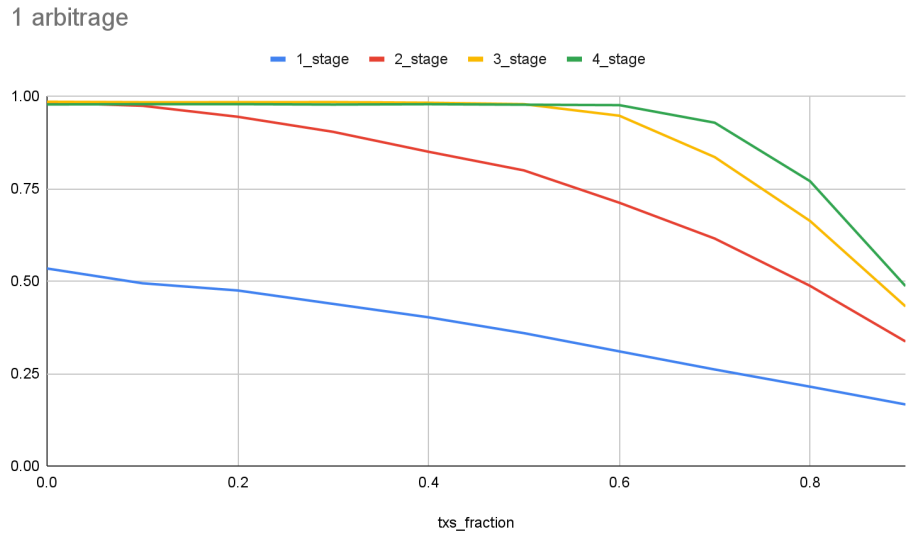
For example, if `read_tx_fraction` is 0.8, then 80% of transactions read the same entry that is also being written by a single transaction.
Benchmark with a single oracle updating the value:



More oracle updates don't reduce the performance and thus are excluded from the doc.

"Arbitrage" scenario

This scenario emulates scenarios similar to arbitrage surge. There are a lot of transaction that have the same RW entry (or a few entries). A similar scenario is an actively used liquidity pool contract.
For the benchmark use the 'baseline' random traffic with relatively low contention and parameters `conflicts_per_tx=5`,`mean_ro_entries_per_conflict=30`,`mean_rw_entries_per_conflict=5`. Then add one more conflict with `txs_fraction` ratio of transactions that write the same key.
For example, if `txs_fraction` is 0.6, then 60% of transactions write the same entry.



More independent ‘arbitrage’ clusters don’t change the results performance significantly and thus are excluded from the doc.

Evaluation results summary

- Adding more stages efficiently works around I/O conflicts, especially for RO-RW conflicts
    - In a lot of scenarios utilization improvement is roughly proportional to the stage ratio (e.g. going from 1 stage to 2 stages leads to 2x higher utilization)
- 3-4 stages allow for almost 100% utilization for lower conflict density cases (with RO-RW density threshold higher than RW-RW density).
    - This is important because there is seemingly no good reason for traffic to have on average high conflict density
- Potential ‘oracle update’ scenario with a lot of transactions reading a key that is being updated is not impactful even when almost all the transactions reading the same key, even with just 2 stages
- Potential ‘arbitrage’ scenario with a lot of transactions writing the same set of keys works well with 3-4 stages, with ~98% utilization even when 60% of the mempool transactions participate in ‘arbitrage’.
    - Obviously, most of the ‘arbitrage’ transactions will not be included in the transaction set as it’s simply impossible to fit more transactions than `max_sequential_instructions` allows.. The key benefit here is that we still let most of the remaining transactions to be executed using the remaining N-1 threads.

Caveats

- The benchmark results showcase building transaction sets from a full mempool (2x of transaction set capacity). Some scenarios definitely have ‘long tail’ execution (e.g.

‘arbitrage’ scenario) and we can’t do much about that from the transaction set building standpoint.
    - If this ever becomes an issue, we’d need to modify the transaction queue in such a way that the number of RW-RW conflicts is below a certain boundary.
- The instructions distribution is rather arbitrary due to the lack of large scale utilization data. E.g. if traffic is biased towards ‘large’ transactions (close to the stage size), then having more stages might reduce the utilization.
    - That’s why everything is configurable - we can adapt to the traffic patterns if necessary in order to reach a compromise between efficiently packing the transactions and working around I/O conflicts